

L04 – Unbalanced (Three-phase) Power Flow

Foundations Lecture Series

Course Instructor

Microgrid 3-phase Security AI Project

Foundations Lecture 4

Keywords: Fortescue transform, sequence networks, VUF, `runpp_3ph`, five violation types

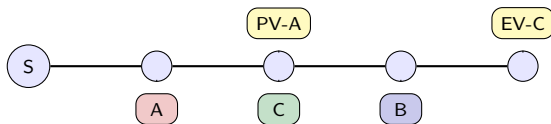
Lecture roadmap

- 1 Where unbalance comes from and what it does in practice
- 2 Symmetrical components: change of basis from phase to sequence
- 3 Sequence networks: three decoupled problems
- 4 Three-phase power flow formulations
- 5 A fully worked example: from single-phase load to VUF
- 6 `pp.runpp_3ph` in pandapower
- 7 Five violation types and the bridge to the project

Prerequisite

L01 (PF basics), L03 (balanced/positive-sequence framing). L02 helpful for the distribution context.

Where does unbalance come from?



Physical sources

- Single-phase residential loads
- Single-phase rooftop PV
- Single-phase EV chargers
- Untransposed LV lines
- Asymmetric line geometry

Why these are unavoidable on LV

- Most homes have a single-phase service drop
- Inverters are wired to the customer's phase
- Transposition is rarely cost-effective at LV
- Loads come and go independently per phase

What unbalance does in practice

Per-phase quantities decouple

- Phase A can be under-voltage while phases B and C are fine.
- One phase of a line can overload while average loading is below 100%.
- Phase angles are no longer exactly 120° apart.

System-level consequences

- Neutral conductor carries current (was zero under balance).
- Increased line and transformer losses.
- Three-phase induction motor heating and torque pulsation.
- Single-phase protection sees different fault levels per phase.

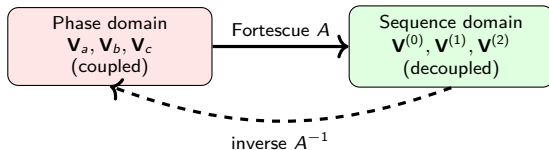
In this course

The microgrid security model must see these per-phase effects. A balanced model would silently report “safe” when phase A is at 0.90 pu. That is exactly the failure mode L04 fixes.

The big idea: change of basis

We have three coupled phase quantities ($\mathbf{V}_a, \mathbf{V}_b, \mathbf{V}_c$) that interact through line and load couplings. The math is tedious.

Symmetrical components change basis to three new quantities ($\mathbf{V}^{(0)}, \mathbf{V}^{(1)}, \mathbf{V}^{(2)}$) that, under a mild symmetry assumption on the *network* (not the loads or sources), are *decoupled* from each other.



Why this is worth doing

Solve three small *decoupled* problems instead of one big coupled one. Then transform the answer back. This is the workflow inside every three-phase PF solver, including pandapower's `runpp_3ph`.

The Fortescue transformation

With $a = e^{j2\pi/3} = -\frac{1}{2} + j\frac{\sqrt{3}}{2}$:

$$\begin{bmatrix} \mathbf{V}^{(0)} \\ \mathbf{V}^{(1)} \\ \mathbf{V}^{(2)} \end{bmatrix} = \underbrace{\frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & a & a^2 \\ 1 & a^2 & a \end{bmatrix}}_{A^{-1}} \begin{bmatrix} \mathbf{V}_a \\ \mathbf{V}_b \\ \mathbf{V}_c \end{bmatrix}, \quad \begin{bmatrix} \mathbf{V}_a \\ \mathbf{V}_b \\ \mathbf{V}_c \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 \\ 1 & a^2 & a \\ 1 & a & a^2 \end{bmatrix}}_A \begin{bmatrix} \mathbf{V}^{(0)} \\ \mathbf{V}^{(1)} \\ \mathbf{V}^{(2)} \end{bmatrix}$$

- $\mathbf{V}^{(0)}$ – **zero sequence**: the common-mode part. All three phases shift in unison.
- $\mathbf{V}^{(1)}$ – **positive sequence**: the balanced rotating ABC part. “True” balanced operation.
- $\mathbf{V}^{(2)}$ – **negative sequence**: balanced rotating ACB (opposite). A pure unbalance signal.

The same transform for currents

$(\mathbf{I}_a, \mathbf{I}_b, \mathbf{I}_c) \leftrightarrow (\mathbf{I}^{(0)}, \mathbf{I}^{(1)}, \mathbf{I}^{(2)})$ with the same matrix. We will apply it to load currents shortly.

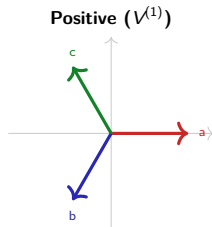
Sequence components of three common patterns

Pattern (phase domain)	$\mathbf{V}_a$	$\mathbf{V}_b$	$\mathbf{V}_c$	Sequence content
Perfectly balanced	V	$a^2 V$	aV	$\mathbf{V}^{(1)} = V$ only
All-A only	V	0	0	$\mathbf{V}^{(0)} = \mathbf{V}^{(1)} = \mathbf{V}^{(2)} = V/3$
Two phases ($V_a = V_b = V, V_c = 0$)	V	V	0	non-trivial $\mathbf{V}^{(0,1,2)}$
Pure unbalance ($V, aV, a^2 V$)	V	aV	$a^2 V$	$\mathbf{V}^{(2)} = V$ only
Zero-sequence only	V	V	V	$\mathbf{V}^{(0)} = V$ only

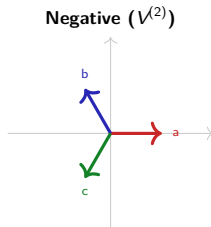
The takeaway

Single-phase load $\rightarrow$ *equal* positive, negative, and zero sequence parts. This is exactly why a single-phase load excites *all three* sequence networks, and why the unbalanced response can be large.

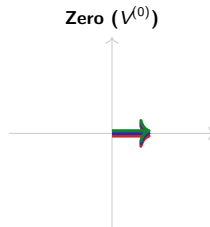
Phasor visualization of the three sequences



ABC, true balance



ACB, asymmetry signal



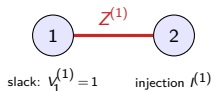
common-mode

What each picture is telling you

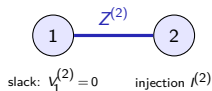
- Positive sequence is what you would draw for a healthy three-phase voltage.
- Negative sequence is the same magnitude rotated the other way – it measures how far we deviate from healthy.
- Zero sequence is three phasors stacked on top of each other – the neutral conductor or ground sees this part.

Three decoupled networks (under symmetric line geometry)

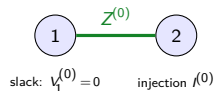
Positive network



Negative network



Zero network



Three things to notice

- For this line-only example the bus graph is the same in all three networks; transformer vector groups and grounding can change the zero-sequence path.
- Only the positive network has a non-zero slack voltage (the generator is balanced).
- Loads inject into all three networks if they are unbalanced; into only the positive if balanced.

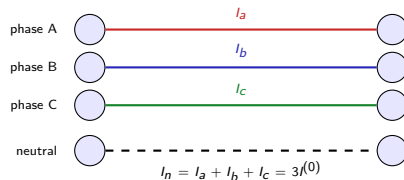
Sequence impedances: where they come from

Element	Typical $Z^{(0)}$ vs $Z^{(1)}$	Why
Transmission line, transposed	$Z^{(0)} \approx 3-5 \cdot Z^{(1)}$	ground/earth return path is high-impedance
LV cable, 4-wire	$Z^{(0)} \approx 2-4 \cdot Z^{(1)}$	neutral conductor + ground return
LV cable, 3-wire (no neutral)	$Z^{(0)}$ very large or undefined	no zero-sequence path
Synchronous generator	machine-specific	neutral grounding and winding data matter
Grounded-wye transformer side	finite if a return path exists	vector group and grounding define the path
Delta winding	blocks zero-sequence line current across that side	zero-sequence current may circulate inside the delta

Practical rule

$Z^{(0)}$ is usually *larger* than $Z^{(1)}$ because zero-sequence current returns through the neutral or earth. That is also why phase-to-ground faults produce different fault currents than three-phase faults. We will use $Z^{(0)} = 2 Z^{(1)}$ in our worked example.

The neutral conductor and zero-sequence



Two equivalent statements

- Under balance, $I_a + I_b + I_c = 0$. No neutral current.
- In sequence terms, $I^{(0)} = (I_a + I_b + I_c)/3 = 0$.

Under unbalance, the neutral carries $3 I^{(0)}$. If the neutral is absent or undersized, the zero-sequence current has nowhere to go and unbalance becomes very expensive.

Why microgrids care

LV microgrid feeders often have a neutral. Single-phase loads cause $I^{(0)} \neq 0$, neutral conductors heat up, and zero-sequence voltage drift can affect every customer on the feeder.

Phase domain vs sequence domain

	Phase-domain PF	Sequence-domain PF
State variables	V_a, V_b, V_c at each bus	$V^{(0)}, V^{(1)}, V^{(2)}$ at each bus
Network coupling	lines have full 3×3 impedance matrices	three decoupled networks
Loads / DER	native phase-domain representation	must transform between domains every iteration
Best for	arbitrary line geometry, untransposed lines	symmetric line geometry, balanced topology with unbalanced injection
Common in	Kersting, OpenDSS	pandapower, fault studies, Grainger

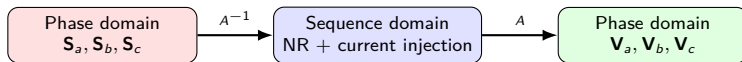
Choosing

If your lines are nicely symmetric (transposed or short LV with similar phase impedances), sequence-domain is cleaner. If geometry is genuinely asymmetric, phase-domain avoids approximation. **This course uses pandapower, so we work in sequence domain.**

pandapower's hybrid approach

`pp.runpp_3ph` solves the three sequence networks at every iteration, with a small twist:

- 1 **Positive network** – solved by full Newton-Raphson. This is the only network with a balanced slack source (the external grid) and PV-type generators.
- 2 **Negative network** – solved by current injection. No internal sources; everything is driven by load and DER unbalance currents.
- 3 **Zero network** – solved by current injection. Same logic, but with the appropriate $Z^{(0)}$ and grounding topology.

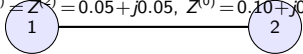


What you see as a user

Just call `pp.runpp_3ph(net)`. The transformations and the three-network solve happen inside.

Setup: 2-bus LV feeder with a single-phase load

balanced slack, $V_1 = 1.00\angle 0^\circ$

$$Z^{(1)} = Z^{(2)} = 0.05 + j0.05, \quad Z^{(0)} = 0.10 + j0.10$$


$$S_a = 0.4 + j0, \quad S_b = S_c = 0$$

All values in per-unit on a 1 MVA / 0.4 kV base.

The load is on phase A only; phases B and C are unloaded. The slack at bus 1 produces a perfectly balanced source. We are about to compute the voltages at bus 2 by hand, then verify with pandapower.

Strategy

- 1 Phase currents at bus 2 $\rightarrow$ sequence currents (Fortescue).
- 2 Sequence voltage drops $\rightarrow$ sequence voltages at bus 2.
- 3 Sequence voltages $\rightarrow$ phase voltages (inverse Fortescue).
- 4 Read off $|V_a|, |V_b|, |V_c|$ and compute VUF.

Step 1 – Phase currents to sequence currents

Flat-start linear approximation: assume $V_2 \approx 1$ so that $I_a = (S_a/V_a)^* \approx S_a$. (The constant-power feedback as V drops is left to the iterative solver; we revisit in Step 4.)

$$I_a \approx 0.4, \quad I_b = 0, \quad I_c = 0.$$

Apply A^{-1} :

$$I^{(0)} = \frac{1}{3}(I_a + I_b + I_c) = \frac{1}{3}(0.4) = 0.1333$$

$$I^{(1)} = \frac{1}{3}(I_a + a I_b + a^2 I_c) = \frac{1}{3}(0.4) = 0.1333$$

$$I^{(2)} = \frac{1}{3}(I_a + a^2 I_b + a I_c) = \frac{1}{3}(0.4) = 0.1333$$

Sequence	Current $I^{(k)}$ (pu)
Zero	0.1333
Positive	0.1333
Negative	0.1333

Observation

A single-phase load excites *all three* sequence networks *equally*. This is why “single phase” is the worst case for unbalance: it produces the maximum amount of negative and zero sequence current per unit of phase current.

Step 2 – Sequence voltage drops at bus 2

Each sequence current flows through its own sequence impedance from slack to bus 2:

$$V_2^{(1)} = V_1^{(1)} - Z^{(1)} I^{(1)} = 1.00 - (0.05 + j0.05)(0.1333) = 0.9933 - j0.0067$$

$$V_2^{(2)} = 0 - Z^{(2)} I^{(2)} = -(0.05 + j0.05)(0.1333) = -0.0067 - j0.0067$$

$$V_2^{(0)} = 0 - Z^{(0)} I^{(0)} = -(0.10 + j0.10)(0.1333) = -0.0133 - j0.0133$$

Sequence	$V_2^{(k)}$ (complex pu)	$ V_2^{(k)} $
Zero	$-0.0133 - j0.0133$	0.0189
Positive	$0.9933 - j0.0067$	0.9934
Negative	$-0.0067 - j0.0067$	0.0094

Sanity

Positive-sequence voltage at bus 2 is 0.993 pu – which would be the answer if this were a balanced 0.133 pu per-phase load. The negative and zero sequences are the “cost” of moving all of it onto phase A.

Step 3 – Sequence voltages to phase voltages (linear estimate)

Apply A (inverse Fortescue):

$$\begin{aligned}\mathbf{V}_a &= \mathbf{V}^{(0)} + \mathbf{V}^{(1)} + \mathbf{V}^{(2)} \\ &= (-0.0133 - j0.0133) + (0.9933 - j0.0067) + (-0.0067 - j0.0067) = 0.9733 - j0.0267\end{aligned}$$

$$\mathbf{V}_b = \mathbf{V}^{(0)} + a^2 \mathbf{V}^{(1)} + a \mathbf{V}^{(2)} = -0.5066 - j0.8726$$

$$\mathbf{V}_c = \mathbf{V}^{(0)} + a \mathbf{V}^{(1)} + a^2 \mathbf{V}^{(2)} = -0.5067 + j0.8594$$

	Phase	$\mathbf{V}_\phi$ (complex pu)	$ \mathbf{V}_\phi $
Magnitudes:	A	$0.9733 - j0.0267$	0.974
	B	$-0.5066 - j0.8726$	1.009
	C	$-0.5067 + j0.8594$	0.998

What the linear estimate is telling us

At this stage hand math says: phase A drops to 0.974, phase B rises slightly to 1.009, phase C stays near nominal. *No phase voltage crosses the $[0.95, 1.05]$ band.* VUF (computed shortly) will also look modest.

But this is the flat-start, first-order answer. Real loads are constant-power, so as $|V_a|$ drops, the actual current grows. We verify on the next slide.

Step 4 – Verify with pandapower (constant-power, iterated)

Running `pp.runpp_3ph` on the same network with the load modeled as constant-power (the default) and

		$ V_a $	$ V_b $	$ V_c $	VUF
iterating to convergence:	Hand math (Step 3, linear flat-start)	0.974	1.009	0.998	0.95%
	<code>pp.runpp_3ph</code> (converged)	0.908	1.029	0.995	3.19%
	Δ (truth – linear estimate)	–0.066	+0.020	–0.003	+2.24%

Why the linear estimate under-reports

As $|V_a|$ drops, a constant-power load draws *more* current ($I = (S/V)^*$). More current produces more voltage drop on all excited sequence networks, which feeds back into a larger V drop. In this stressed example, the iterative solution therefore differs materially from the flat-start estimate.

Take-away

Linear hand math gives intuition. **For safety screening, always use the iterative solver.** The project's Week 4 labeling code is exactly this: `runpp_3ph` on every scenario $\times$ contingency.

VUF and a common magnitude-deviation metric

IEC 61000-4-30 (true VUF)

$$\text{VUF} = \frac{|\mathbf{V}^{(2)}|}{|\mathbf{V}^{(1)}|} \times 100\%$$

The ratio of negative-sequence to positive-sequence voltage magnitude.

NEMA MG 1 percent voltage unbalance (not the same metric)

$$\% \text{VU}_{\text{NEMA}} = 100 \frac{\max_{xy \in \{ab, bc, ca\}} |V_{xy}| - \bar{V}_{LL}}{\bar{V}_{LL}}, \quad \bar{V}_{LL} = \frac{|V_{ab}| + |V_{bc}| + |V_{ca}|}{3}.$$

It uses line-to-line voltage magnitudes; it is a useful motor-oriented proxy, but it is not algebraically equivalent to sequence VUF.

Reference practice	Metric and teaching reference
IEC 61000-2-2 (public LV compatibility)	sequence VUF: 2%
NEMA MG 1 (motor terminals)	percent voltage unbalance: 1% recommended maximum

VUF on our worked example

From hand math (Step 2):

$$|\mathbf{V}_2^{(1)}| = |0.9933 - j0.0067| \approx 0.9934$$

$$|\mathbf{V}_2^{(2)}| = |-0.0067 - j0.0067| \approx 0.0094$$

$$\text{VUF}_{\text{linear}} = \frac{0.0094}{0.9934} \times 100\% \approx \mathbf{0.95\%} \quad (\text{below 2\% limit - no violation in linear estimate})$$

From pandapower (Step 4):

$$\text{VUF}_{\text{converged}} \approx \mathbf{3.19\%}$$

This is above both the IEC 2% limit *and* the ANSI 3% limit. The converged state has two simultaneous violations:

- Phase-A undervoltage ($|V_a| = 0.908 < 0.95$).
- VUF excursion ($3.19\% > 2\%$).

Why the project tracks VUF separately

Even when no individual phase voltage crosses its limit, a high VUF damages three-phase motors and trips sensitive equipment. The Week 4 labeling code treats VUF as a stand-alone violation type, computed from `res_bus_3ph.unbalance_percent`.

Asymmetric loads and sources

```
import pandapower as pp

net = pp.create_empty_network(sn_mva=1.0)
b1 = pp.create_bus(net, vn_kv=0.4)
b2 = pp.create_bus(net, vn_kv=0.4)
pp.create_ext_grid(net, bus=b1, vm_pu=1.0,
    s_sc_max_mva=1e6, rx_max=0.1,
    x0x_max=1.0, r0x0_max=0.1)
pp.create_line_from_parameters(net, b1, b2, length_km=1,
    r_ohm_per_km=0.008, x_ohm_per_km=0.008, c_nf_per_km=0.0,
    r0_ohm_per_km=0.016, x0_ohm_per_km=0.016, c0_nf_per_km=0.0,
    max_i_ka=10.0)

# Single-phase load: 0.4 pu on phase A, zero on B and C (1 MVA base)
pp.create_asymmetric_load(net, bus=b2,
    p_a_mw=0.4, q_a_mvar=0.0,
    p_b_mw=0.0, q_b_mvar=0.0,
    p_c_mw=0.0, q_c_mvar=0.0,
    type="wye")
```

Key new parameters

- `r0_ohm_per_km`, `x0_ohm_per_km`: zero-sequence line parameters.
- `create_asymmetric_load`: explicit per-phase P , Q , plus wye/delta type.
- `create_asymmetric_sgen`: same idea for PV or BESS on one phase.

Running and reading the three-phase result

```
pp.runpp_3ph(net)

print(net.res_bus_3ph[["vm_a_pu", "vm_b_pu", "vm_c_pu",
                      "unbalance_percent"]])
print(net.res_line_3ph[["loading_a_percent",
                        "loading_b_percent",
                        "loading_c_percent",
                        "i_n_from_ka"]])
```

Expected output for the worked example (rounded):

bus	vm_a_pu	vm_b_pu	vm_c_pu	unbalance_percent
1	1.000	1.000	1.000	0.00
2	0.908	1.029	0.995	3.19

Numbers match Step 4, not Step 3

`runpp_3ph` solves the constant-power, iterated problem and matches the converged values from Step 4. The Step-3 linear hand math underestimated the violation. Always cross-check.

Note on solver options

`pp.runpp_3ph` has its own internal solver (NR on the positive network, current injection on negative and zero). It does *not* accept `algorithm="bfs"` – BFSW is a balanced-PF option for `pp.runpp` only.

What the project labels as a “violation”

The supervised dataset built in Week 4 of the project assigns a binary label *violation* that is the logical

OR of five sub-flags:

#	Violation type	Detected from
1	Phase under- or over-voltage	<code>vm_a_pu</code> , <code>vm_b_pu</code> , <code>vm_c_pu</code> vs $[0.95, 1.05]$
2	Line / transformer phase overload	<code>loading_a/b/c_percent</code> vs 100%
3	Voltage unbalance factor	<code>unbalance_percent</code> vs 2%
4	Service loss	topology check: are loads still connected?
5	Non-convergence	<code>net.converged_3ph</code> flag

The pattern

The first three are read straight out of `runpp_3ph`'s result tables. The fourth is a graph reachability check on the post-contingency network. The fifth is the solver's own status flag.

Why this lecture is the foundation of the project

Every piece of the project relies on L04

- Week 3 generates operating scenarios with *asymmetric* loads and DER.
- Week 4 runs `runpp_3ph` on every scenario $\times$ contingency.
- Week 4 reads phase voltages, phase loadings, VUF.
- Week 5–7 train AI models to predict the label.
- Week 8 evaluates recall on these specific violation types.

If you only remember one thing

A balanced solver cannot answer microgrid security questions on an LV feeder. Every result that depends on per-phase quantities must come from `runpp_3ph` or an equivalent unbalanced engine.

This is the rigour the AI model is asked to inherit

The AI screens these labels. If the AI model is to be trusted, the labels must be correct – and “correct” here means *exactly the kind of unbalanced analysis we just did by hand*.

Homework

Required

- 1 Reproduce the 2-bus worked example in pandapower. Verify $|V_a|, |V_b|, |V_c|$ at bus 2 match the slides to 3 decimals.
- 2 Compute VUF from `res_bus_3ph[``unbalance_percent']` and confirm it equals $\approx 3.19\%$.
- 3 Sweep the phase-A load from 0.05 pu to 0.65 pu in steps of 0.05. Plot $|V_a|, |V_b|, |V_c|$ on the same axes. Find the smallest load that violates *any* of the five label types. Note where `runpp_3ph` stops converging.
- 4 Replace the phase-A load with an `asymmetric_sgen` of 0.4 pu on phase A (i.e., single-phase PV instead of single-phase load). Recompute. Which phase now sees overvoltage?

Optional

- 1 Vary $Z^{(0)}/Z^{(1)}$ from 1 to 10. Plot how $|V_a|$ at bus 2 changes for a fixed phase-A load. Explain.
- 2 Repeat (1) with a delta-connected asymmetric load. How does the VUF behave?

Exit checklist

- 1 ✓ I can list the main physical sources of unbalance on an LV feeder.
- 2 ✓ I can write the Fortescue matrix A^{-1} from memory and explain what each row computes.
- 3 ✓ I can sketch the three sequence networks for a given physical system.
- 4 ✓ I can argue why $Z^{(0)} > Z^{(1)}$ in a typical 4-wire feeder.
- 5 ✓ I can take phase currents $\rightarrow$ sequence currents $\rightarrow$ sequence voltages $\rightarrow$ phase voltages, by hand, on a small example.
- 6 ✓ I can distinguish IEC sequence VUF from NEMA percent voltage unbalance.
- 7 ✓ I can call `pp.runpp_3ph` and read `res_bus_3ph`.
- 8 ✓ I can list the five violation types the project labels.

References

- **Fortescue, 1918.** *Method of Symmetrical Co-ordinates Applied to the Solution of Polyphase Networks*, AIEE Trans. The founding paper.
- **Grainger & Stevenson,** *Power System Analysis*. Chapters on symmetrical components, sequence networks, and unbalanced faults.
- **Kersting,** *Distribution System Modeling and Analysis* (4th ed.). The phase-domain treatment used in OpenDSS.
- **Anderson,** *Analysis of Faulted Power Systems*. Deep treatment of sequence-component fault analysis.
- **IEC 61000-4-30:2015,** *Testing and measurement techniques – Power quality measurement methods*. The IEC VUF definition.
- **ANSI C84.1-2020,** *Electric Power Systems and Equipment – Voltage Ratings*. Voltage and unbalance limits.
- **pandapower docs:** “Asymmetric / Three-phase Power Flow” page and the `runpp_3ph` reference.